

Project Option — GPU Wavelets (JPEG2000-style MRA)

GPU Computing (Master), bonus project

Goal. Starting from the provided 1D CPU reference (`1D_wavelets.cpp`), implement a fully GPU-accelerated wavelet transform and a simple multi-resolution representation (MRA) in the style of JPEG2000. Develop from a minimal 1D baseline (analysis + synthesis) into an efficient 2D GPU implementation.

General Project Guidelines

The following guidelines apply to all projects.

Scope & Spirit

Projects are meant to deepen your understanding of GPU computing through a concrete, measurable task. You may **deviate** from the suggested items if you have a better idea, provided it stays within the *spirit* of the project (same learning goals). If you wish to propose a new scoring item not listed, **ask for approval** first; I will grant it if it has clear pedagogical value.

Deliverables

1. **Repository (required).** CUDA/C++ code with a clear structure and clear build/run instructions (e.g., in a `README.md`). Ideally, an automated build system (e.g., `CMakeLists.txt`).
2. **Report (required).** Either *Markdown* (in the repo) or a *PDF* (3–10 pages). It should focus on:
 - What you implemented (which items, with brief descriptions).
 - Performance discussion: methodology, measurements, what limits throughput.
 - Design/implementation choices: data layout, synchronization methods, trade-offs.
 - Anything worth a discussion (e.g., failed attempts, unexpected findings).

Please note that the report is as important as the code for the assessment.

Submission & Deadlines

Deadlines

Mid-semester feedback (optional): *July 17, 2026* (Sun), 23:59 CET.

Hard deadline: *July 31, 2026* (Sun), 23:59 CET.

Mid-semester feedback (optional). Submit a draft by July 17 to get written feedback from me:

- Are you on track to reach the threshold (bonus) if submitted as is?
- Are you heading in a sensible technical direction?
- Any major flaw or mismatch with course goals?
- If your chosen direction is off-topic for the course, I will say so here.

Submission Workflow

1. Host the code on the LRZ GitLab. **Fork** the course repo (or create your own), open a **Merge Request**, and **assign me as reviewer**. If anything special should be noted you can add a short description in the MR.
2. If you do not have LRZ GitLab access, email me a link to your repository.

Evaluation

- **Grading.** Evaluation is *pass/fail* based on the sum of points from the project's item list. Each item has a stated value; you can earn *full*, *partial*, or *no* credit depending on the quality of your implementation and evidence. If your total is ≥ 50 points, you **pass** and receive a **-0.3** bonus on the final grade; otherwise, no bonus is awarded. The projects' maximum amount of points **exceeds** 50 and you do **not** need to implement everything and should rather pick the items that best fit your interests and time. Please mention in the report which items you implemented. Items completed with more depth or quality may receive *extra* points at my discretion.
- **Workload expectation.** The bonus is calibrated for ≈ 20 hours of effective work *per student*. Your submission should reflect this depth (code, measurements, and report).
- **What I value.** Code correctness first, then correct reasoning, then performance. Please note that it is still interesting if an optimization attempt *fails* to deliver speedup, moreso if you can explain *why*.
- **Evidence.** Include timing tables, plots, or other measurements for all relevant items. Detail as much as possible your measurement methodology (e.g., warmup, averaging, input sizes). Performance-related items will not be awarded points without evidence.

General Rules

- Your work will be assessed individually. You may collaborate with classmates for discussion, but the code and report must be your own.
- The use of AI tools is permitted but you will be held accountable for your submission.
- Unless specified otherwise, you are expected to propose pure CUDA kernels without using external libraries (e.g., Thrust, cuBLAS). Occasional, well-justified library calls are acceptable; in particular, in-kernel CUB collectives (e.g., `cub::WarpScan`, `cub::BlockScan`, `cub::BlockReduce`) are permitted. Avoid relying on host-side opaque primitives (e.g., `thrust::exclusive_scan`, `cub::DeviceScan`) for your solution, although it is recommended to compare against them when appropriate (and it is perfectly fine to be slower). The objective of this project is to demonstrate kernel-level design (memory hierarchy, synchronization, divergence).

Quick Checklist

- The *readme* or *report* explains how to compile and run your code.
- Following the instructions indeed builds and runs the code.
- There is a report (Markdown or PDF) covering implementation, measurements, and analysis.
- Submission via GitLab MR (assigned to me) or, if unavailable, by email with a link.

Introduction

Wavelet transforms are a broad family of techniques that decompose signals into different scale components. Separable wavelet transforms are widely used in image processing and compression

(e.g., JPEG2000) because they include useful properties that we will not detail here. One such property is the ability to perform multi-resolution analysis (MRA), which decomposes an image into a hierarchy of frequency subbands at multiple scales.

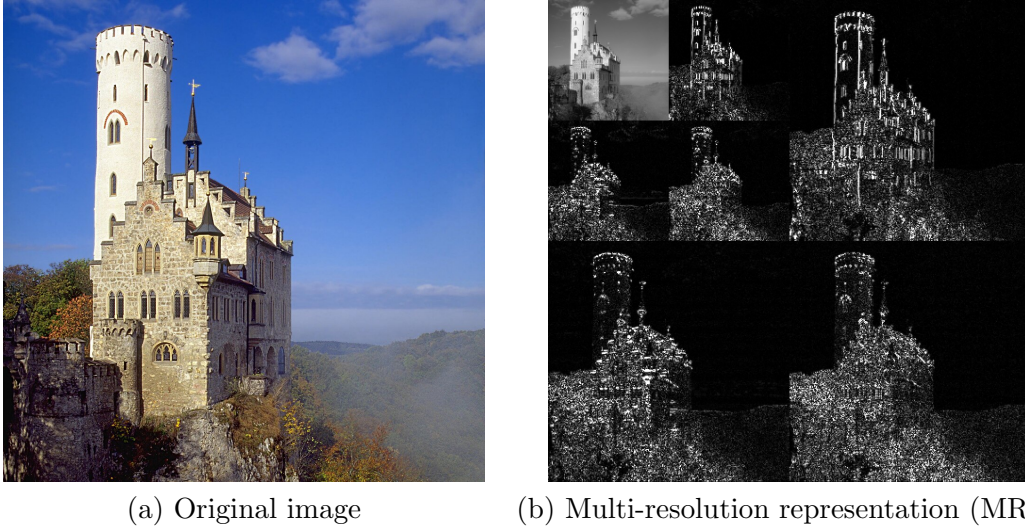


Figure 1: (a) Original image and (b) its multi-resolution representation via wavelets. By applying the same transform recursively (MRA), we obtain a hierarchical decomposition of the image into different frequency subbands at multiple scales.

In Figure 1, we see an example of an image and its MRA representation obtained via separable wavelets. To obtain this image, we can apply the following steps:

1. **1D transform.** Implement the forward and inverse wavelet scheme in one dimension (1D). This 1D transform is provided in the baseline code `1D_wavelets.cpp`.
2. **2D separable transform.** By applying the 1D transform first along all the rows and then along the columns of an image, we obtain a 2D image with 4 quadrants (subbands): LL (low-low), HL (high-low), LH (low-high), and HH (high-high). The LL subband contains a coarse approximation of the image, while the other three subbands capture different orientations of detail. *Note:* The order can also be columns first, then rows; the result is the same.
3. **Multi-level MRA.** By recursively applying the 2D separable transform to the LL subband, we can build a multi-level MRA pyramid. Each level of the pyramid represents the image at a coarser resolution, capturing progressively lower frequency content.

This process is reversible: applying the inverse transform in reverse order reconstructs the original image from its MRA representation. Moreover, it can be parallelized on GPUs: some parts are evidently parallel (e.g., row/column transforms), but careful synchronization and memory access patterns are needed to achieve correct and efficient executions.

Baseline Provided

The only baseline provided is:

- **1D_wavelets.cpp:** A minimal CPU reference that shows how the transform works in 1D. It implements:
 - a forward transform (referred to as *analysis* in signal processing): base signal to wavelet coefficients,
 - an inverse reconstruction (referred to as *synthesis*): wavelet coefficients back to base signal.
 - a basic test on an arbitrary 1D signal to validate correctness.

Building and running the baseline.

```
g++ -O2 1D_wavelets.cpp -o wavelet1D
./wavelet1D
```

Scoring Rubric

1. **1D on GPU (FWD+INV) (10 pts).**
Implement forward and inverse transforms in 1D on the GPU. Validate against the CPU baseline for multiple signal sizes.
2. **2D separable transform (10 pts).**
Implement 2D forward and inverse transforms via separable row/column passes. Validate perfect reconstruction on a small synthetic image.
3. **Multi-Resolution Analysis (MRA) (15 pts).**
Recursively process the LL band. The process should still yield perfect reconstruction: applying N levels of FWD followed by N levels of INV returns the original image. Beware of changing indexing and strides when jumping to deeper levels.
4. **Image I/O (5 pts).**
Load/save images (e.g., PNG, PGM, custom format, etc.). Use external libraries if needed (e.g., stb_image.h).
5. **Data layout (10 pts).**
Maybe the default row-major layout is not ideal for multi-level MRA. Discuss alternatives (e.g., Morton order, tiled layout), (eventually) implement different ones, and compare performance. Finding a clear winner is not expected, but a good discussion is.
6. **Coalesced row/column passes (10 pts).**
Ensure row passes use coalesced memory. For column passes, avoid naive strided loads (e.g., tiled transpose or shared memory buffering). Show a bandwidth comparison (before/after).
7. **Shared-memory tiling (15 pts).**
Use shared-memory tiles to perform wavelet transforms within each tile. The overall result should not necessarily be the same (tricky to do because of boundaries); the goal is rather to reduce global memory traffic and see if performance improves.
8. **Multi-resolution access kernel (20 pts).**
One of the benefits of MRA is efficient access to different resolutions. For instance, when analyzing a zoomed-out view of a gigantic image, it is more reasonable to access only the coarsest levels of the MRA pyramid. Assuming all the MRA-related data are resident on the GPU, implement a kernel that extracts a given part at a given resolution level and sends it back to the host.
9. **3D extension (10 pts).**
Extend your implementation to 3D data (e.g., volumetric images).
10. **FP16 path (5 pts).**
Support FP16 inputs/outputs and do the transform in FP16 with reasonable performance (beware bank conflicts, etc.).
11. **Performance study (10 pts).**
Present bandwidth, occupancy, register count, and limiting factors in different configurations (image sizes, levels, data types). Optionally, present a roofline analysis.
12. **CI + reproducibility (10 pts).**
Provide a simple CI setup that builds the project and runs small deterministic tests (e.g., 1D and 2D transforms on tiny signals/images), checking basic invariants (no NaNs, near-perfect reconstruction).
13. **Command-line interface (CLI) (5 pts).**
Provide a simple CLI to specify input/output files, number of levels, data type, and other parameters.

14. **Code quality & tests (10 pts).**

Provide unit tests on different cases/sizes. Keep kernels and host code modular and documented.